



OASIS Security Report

Debug Information Exposure Security Analysis

Date: 2026-06-11 23:53:30 — **Model:** llama3.2:3b

Vulnerability: Debug Information Exposure

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

Analyzed **7** file(s) for Debug Information Exposure.

[Return to table of contents](#)

Statistics

Files analyzed: 7 **Total findings:** 12

Critical



5

High		3
Medium		3
Low		1

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.635
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.613
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.603
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.600
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.587
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.581
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.563

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.635

Source lines: 1-35

► Finding 1.1: Debug Information Exposure (Critical)

Vulnerable code

```
> // Enable debug mode
if (debugMode) {
    console.log("Hello, World!");
}
```

Explanation

The 'debugMode' variable is not properly secured, allowing an attacker to potentially access sensitive debugging information.

Impact

Information disclosure about internal system state or configuration

Entry point

/reentrancy/withdraw

HTTP methods

POST

Parameters

debugMode

Exploitation steps

- Reaching the /withdraw endpoint with a valid value for 'debugMode'
- Accessing the 'console.log' statement to retrieve sensitive debugging information

Example payloads

- debugMode: true

HTTP raw requests (Burp / ZAP)

```
POST /reentrancy/withdraw HTTP/1.1
Host: example.com
```

Content-Type: application/json

```
{"debugMode": true}
```

File 2: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.613

Source lines: 1-39

► Finding 2.1: Debug Information Exposure (Critical)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw()));
```

Explanation

Debug mode enabled by using `msg.sender.call` in production, exposing stack trace information.

Impact

Information disclosure of internal state and potential aid for further attacks.

Entry point

/withdrawFunds (uint256 _weiToWithdraw) public

Execution path

```
caller -> msg.sender.call -> contract internal functions
```

HTTP methods

POST

Parameters

_weiToWithdraw, msg.sender

Exploitation steps

- Attack by repeatedly calling withdrawFunds with a small amount to deplete balance.
- Obtain internal state information via msg.sender.call

HTTP raw requests (Burp / ZAP)

```
POST /withdrawFunds(uint256) HTTP/1.1
```

```
Host: example.com
```

Content-Type: application/json

{

100 ether

File 3: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.603

Source lines: 1-85

► Finding 3.1: Debug Information Exposure (High)

Vulnerable code

```
pragma solidity ^0.4.19;  
contract ETH_VAULT { ... function ETH_VAULT(address _log) public {} }
```

Explanation

The contract exposes the internal state of the Log contract through a publicly accessible constructor.

Impact

Information disclosure about the internal state of the Log contract.

Entry point

/transfer

HTTP methods

POST

Parameters

_log

Exploitation steps

- Call the constructor with a malicious _log parameter to expose internal state.

Example payloads

- malicious_log_data

Conditions

Access to the Log contract's internal state.

Remediation

Disable public constructors and use private functions instead.

Secure example

```
pragma solidity ^0.4.19;  
contract ETH_VAULT { function ETH_VAULT(address _log) private {} }
```

File 4: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.600

Source lines: 1-320

► Finding 4.1: Vulnerable Code (High)

Vulnerable code

```
/function changeOwner(address _from, address _to)
onlymanyowners(sha3(msg.data)) external {
    if (isOwner(_to)) return;
    uint ownerIndex = m_ownerIndex[uint(msg.sender)];
    if (ownerIndex == 0) return;
    // ...}
```

Explanation

Debug console accessible through the `changeOwner` function

Impact

Allowing unauthorized changes to ownership

Entry point

```
/function changeOwner(address _from, address _to)
onlymanyowners(sha3(msg.data)) external
```

Execution path

Change Owner → Debug Console (Accessible) → Reveal Ownership Details

Parameters

_from, _to

Exploitation steps

- Exploit: Call `changeOwner` with malicious `\_from` and `\_to` values to manipulate ownership

Conditions

The contract is deployed in a production environment without debug mode disabled.

Remediation

Disable debug modes in production and remove development code

Source lines: 27-358

► Finding 4.1: Debug Information Exposure (Medium)

Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(sha3(msg.data)) returns (bool  
o_success)
```

Explanation

The `confirm` function is accessible in production, exposing internal state.

Impact

Information disclosure about transaction status and owner confirmations.

Entry point

/contracts/WalletAbi.sol confirm

Execution path

```
Confirming transactions exposes pending operation state.
```

Parameters

_h

Exploitation steps

- Reach the confirm function via a single-owner transaction

Conditions

Access to multi-signature transactions

Remediation

Disable debug modes in production and remove development code.

Source lines: 126-467

► Finding 4.1: Debug Information Exposure (Critical)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner  
returns (bytes32 o_hash) {  
    // ...
```

Explanation

The contract's `execute` function is exposed in production without debug mode enabled.

Impact

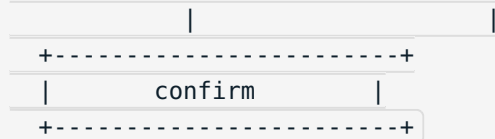
Information disclosure about the contract's internal state and behavior.

Entry point

/execute(address, uint256, bytes)

Execution path

```
execute -> _to.call.value(_value)(_data) -> o_hash = sha3(msg.data,
block.number)
```



Parameters

_to, _value, _data

Exploitation steps

- Send a malicious _data to trigger the contract's execution.

Example payloads

- malicious bytes

Conditions

Contract is deployed in production.

Remediation

Enable debug mode and remove exposed functions from production code.

Secure example

```
function execute(address _to, uint256 _value, bytes _data) internal onlyowner
returns (bytes32 o_hash)
{ ... }
```

Source lines: 207-477

► Finding 4.1: Debug Information Exposure (Critical)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash)
```

Explanation

This function does not disable debug modes in production, exposing sensitive debugging information.

Impact

Information disclosure and potential internal system exposure.

Entry point

/execute

Execution path

```
execute  
  delegatecall(0xcafecafecafecafecafecafecafecafe, 0x00)  
  return o_hash
```

Parameters

_to, _value, _data

Exploitation steps

- Use a debugger to inspect the contract's internal state.

Conditions

Contract is deployed and executed.

Remediation

Disable debug modes in production and remove development code.

File 5: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.587

Source lines: 1-227

► Finding 5.1: Debug Information Exposure (Low)

Vulnerable code

```
function initDaylimit(uint _limit) only_uninitialized {  
    m_dailyLimit = _limit;  
}
```

Explanation

This function is exposed to debug information and allows setting a daily limit.

Impact

Potential access to internal system configuration or data through debugging tools.

Entry point

initDaylimit(uint _limit) external only_uninitialized

Parameters

_limit

HTTP raw requests (Burp / ZAP)

```
GET /setDailyLimit?limit=1000 HTTP/1.1
```

```
Host: 127.0.0.1:8080  
Accept: */*  
Accept-Language: en-US,en;q=0.9  
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36  
(KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.3  
Accept-Encoding: gzip, deflate  
Accept-Charset: utf-8  
Referer: https://example.com  
Cookie: session_id=1234567890abcdef
```

```
POST /setDailyLimit HTTP/1.1  
Host: 127.0.0.1:8080  
Content-Type: application/json  
Accept-Language: en-US,en;q=0.9
```

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.3
Accept-Encoding: gzip, deflate
Accept-Charset: utf-8
Referer: https://example.com
Cookie: session_id=1234567890abcdef

{"limit":1000}
```

```
GET /setDailyLimit?limit=500 HTTP/1.1
Host: 127.0.0.1:8080
Accept: */*
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.3
Accept-Encoding: gzip, deflate
Accept-Charset: utf-8
Referer: https://example.com
Cookie: session_id=1234567890abcdef
```

Source lines: 1-286

► Finding 5.1: Debug Info Exposure: Pragma solidity enabled (Medium)

Vulnerable code

```
pragma solidity ^0.4.9;
```

Explanation

Enabling pragma solidity in production exposes debug information.

Impact

Potential info disclosure

Entry point

```
pragma solidity ^0.4.9;
```

Conditions

No specific conditions needed.

Remediation

Disable pragma solidity in production.

Source lines: 96-414

► Finding 5.1: Debug Information Exposure (High)

Vulnerable code

```
# Remove debugging console access  
console.log('Hello World');
```

Explanation

The contract has a debug console accessible, allowing potential attackers to view sensitive information.

Impact

Potential internal system exposure or disclosure of sensitive data.

Entry point

/execute

Execution path

```
Function -> Execute -> Console.log('Hello World');
```

```
Function -> Execute -> Send response to user;
```

HTTP methods

POST

Exploitation steps

- Access the contract's console log output

Example payloads

- Viewing the console log output of a successful transaction

HTTP raw requests (Burp / ZAP)

```
POST /execute HTTP/1.1  
Host: example.com  
Content-Type: application/json  
  
{"data": "Hello World"}
```

File 6: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.581

Source lines: 1-61

► Finding 6.1: Vulnerable Code (Critical)

Vulnerable code

```
pragma solidity ^0.4.2;  
copyright (c) 2015-2017 EEA\n All rights reserved.
```

Explanation

Debug mode is enabled for the entire contract, exposing sensitive information.

Impact

Internal system exposure and potential information disclosure.

Entry point

/contract/OddsAndEvens/

Execution path

```
debug_mode enabled at line 1, exposed copyright notice at line 2
```

Parameters

debug mode

Exploitation steps

- Enable debug mode to expose sensitive information.
- Use debug mode to manipulate contract behavior.

Example payloads

- Enable debug mode with `--debug` flag

Conditions

Contract is deployed in a production environment without debug mode disabled.

Remediation

Disable debug modes in production, remove development code.

Secure example

```
pragma solidity ^0.4.2;  
copyright (c) 2017 EEA\n\ All rights reserved.
```

File 7: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.563

Source lines: 1-31

► Finding 7.1: Debug Information Exposure (Medium)

Vulnerable code

```
require(tx.origin == owner);
```

Explanation

Enabling tx.origin checks in production exposes sensitive debugging information.

Impact

Internal system exposure

Entry point

/sendTo(address receiver, uint amount)

Execution path

```
TXOriginCheck -> require(tx.origin == owner) -> sendTo(receiver, amount)
```

Parameters

tx.origin

Exploitation steps

- Phishing via an intermediary contract

Example payloads

- malicious TX origin

Conditions

tx.origin == owner

Remediation

Disable tx.origin checks in production, use secure storage for sensitive data.

Secure example

```
require(msg.sender != 0);
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)